PRODUCT REQUIREMENTS DOCUMENT

# Enterprise Feature Store Discovery UI

*Scoped to the Model Builder persona — v1.0*

| AUTHOR | ROLE | DATE |
|---|---|---|
| Sachin Doddamani | Senior Product Manager | October 2026 |
| **VERSION** | **STATUS** | **HORIZON** |
| 1.0 — Initial Draft | In Review | H2 Roadmap |

CONTENTS


01 Executive Summary
02 Problem Statement
03 Background & Market Context
04 Goals & Non-Goals
05 Success Metrics
06 User Personas
07 Jobs to be Done
08 Solution Overview
09 Feature Requirements
10 User Stories
11 Non-Functional Requirements
12 Out of Scope
13 Dependencies & Risks
14 Go-to-Market & Launch Plan
15 Open Questions

## 01 Executive Summary

The Enterprise Feature Store (EFS) is a governed, reusable feature layer that sits at the heart of the AI/ML lifecycle — bridging data preparation and model training. Today, EFS is a powerful Python library used by a small group of data scientists who know the API. It has no visual interface, no self-service discovery surface, and no way for a Model Builder to explore, understand, or assemble training data without writing code or asking a data engineer for help.

This PRD defines the product requirements for the **EFS Discovery UI**, a first-class web interface scoped to the **Model Builder persona**. The UI enables a Model Builder to browse the feature catalog, understand feature quality through lineage and freshness signals, compose a point-in-time-correct training dataset from discovered features, and hand it off to a model training pipeline, all without writing a single line of Python.

> **THE CORE BET**
>
> Turning EFS from a Python library into a self-service product will unlock the Model Builder persona as a first-class user, drive measurable increases in feature reuse across teams, and close a capability gap that competing platforms already cover.

The expected outcome is a 60% reduction in time-to-training-dataset for Model Builders, a meaningful lift in cross-team feature reuse, and a platform that positions EFS as enterprise-ready infrastructure rather than an engineering utility.

## 02 Problem Statement

### CURRENT STATE

EFS ships today as a Python SDK. To use it, a Model Builder must know the exact feature names, understand the underlying schema, write transformation logic from scratch, and coordinate directly with the data engineering team to understand what is available. There is no catalog, no search, and no visual interface.

The consequence is predictable: **features are rebuilt rather than reused**. Duplicate feature pipelines exist across teams. Training datasets are assembled ad hoc, with lineage lost and reproducibility impossible. Models trained on stale or misunderstood features fail in production in ways that are difficult to diagnose.

### THE USER'S PAIN

A Model Builder working on a fraud detection model today must file a ticket with the data engineering team to ask what customer behavioural features exist, wait for a response, receive a list of table names with no context, write Python to inspect schemas, discover that two features look similar but have different time windows, and ultimately rebuild one from scratch because the existing version cannot be trusted without provenance information. This process routinely takes days for what should take minutes.

### WHY THIS MATTERS NOW

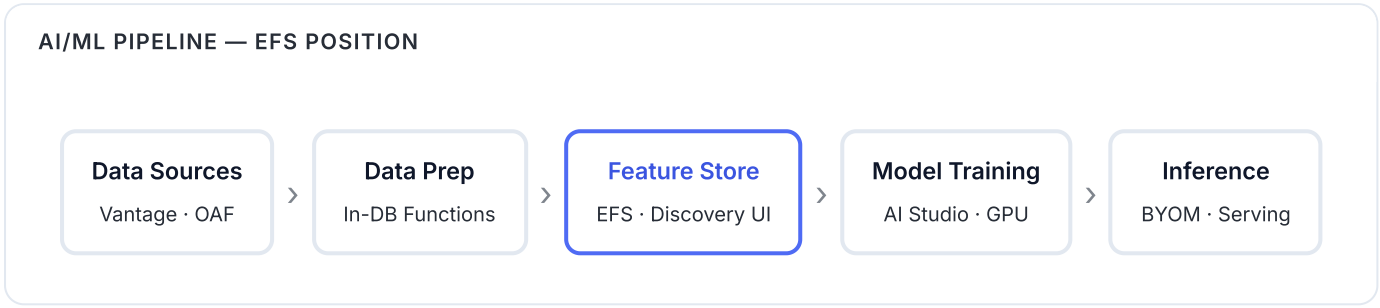
Industry data shows that 85% of ML models never reach production. Poor feature engineering hygiene — features rebuilt, lineage lost, training data not reproducible — is one of the leading structural causes. At the same time, Databricks acquiring Tecton in August 2025 for approximately $900M signals that feature stores are no longer niche infrastructure. Competitors now offer visual UIs. EFS does not. The gap is a known Horizon-2 roadmap item and represents a measurable competitive risk.

> **VALIDATED CUSTOMER SIGNAL**
>
> Sanofi, an active EFS deployment, has explicitly identified natural-language access to the EFS knowledge base as one of four top product priorities. Business users have limited visibility into data products, and EFS's Python-only interface is the primary barrier. BNP Paribas and Vanguard have independently engaged on feature store discovery tooling.

## 03 Background & Market Context

| $90B | 40%+ | 85% | ~$900M |
|---|---|---|---|
| MLOps market size by 2034 | Market CAGR | ML models that never reach production | Databricks acquisition of Tecton (Aug 2025) |

### WHERE EFS FITS IN THE AI/ML LIFECYCLE




EFS is the spine of the ML lifecycle. One governed feature layer feeds both training — through point-in-time-correct dataset assembly — and inference, through the same features served for batch and online scoring. The Discovery UI unlocks this value for the Model Builder who consumes features but does not author them.

### WHY A UI, WHY NOW

A UI transforms EFS from a powerful engineering library into a self-service enterprise product. Without one, discovery stays locked in code and tribal knowledge. Three forces align to make this the right moment: the competitive landscape has moved (Tecton, Feast, SageMaker Feature Store all have UIs), the customer base is signalling clear demand (Sanofi, BNP Paribas, Vanguard), and the platform's existing Horizon-2 roadmap identifies Discovery UI as a known gap.

## 04 Goals & Non-Goals

### GOALS

**G1 — Enable self-service discovery.** A Model Builder can find, evaluate, and select features from the catalog without writing code or involving a data engineer.

**G2 — Increase feature reuse.** Surfacing existing features with rich context (lineage, freshness, usage) makes reuse the path of least resistance over rebuilding.

**G3 — Accelerate time to training dataset.** From identifying relevant features to having a versioned, point-in-time-correct dataset ready for model training, the end-to-end flow should complete in minutes, not days.

**G4 — Expand the EFS user base.** Bring analysts, business-aligned data scientists, and less Python-fluent ML practitioners into EFS as first-class users alongside the existing engineering audience.

**G5 — Build governance by design.** Every dataset produced through the UI is traceable, versioned, and reproducible — lineage and provenance come with the workflow, not as an afterthought.

### NON-GOALS

**NG1 — Feature authoring.** Creating new features, writing transformation logic, and managing feature pipelines are Feature Engineer workflows. This PRD covers consumption, not production.

**NG2 — Platform administration.** Repository setup, user access management, schema governance, and SLA configuration are Platform Admin workflows and are explicitly out of scope for this release.

**NG3 — Python SDK changes.** The Discovery UI is a new surface built on the existing SDK. SDK-level changes, new API endpoints, and SDK feature parity are engineering concerns addressed separately.

**NG4 — Online serving and inference.** Configuring features for real-time serving, managing online store deployments, and monitoring inference latency are post-training concerns outside this UI's scope.

**NG5 — Cross-platform connectors.** Integrations with external ML platforms (SageMaker, Vertex AI, Azure ML) are on a separate roadmap track and not part of this release.

## 05 Success Metrics

### PRIMARY METRICS

**Time to training dataset (T2TD).** Median elapsed time from a Model Builder opening the UI to exporting a validated, point-in-time-correct dataset to their training pipeline. Baseline: 2 to 3 days (current ticket-driven process). Target: under 30 minutes at GA. This is the headline metric and the most direct signal of whether the UI is doing its job.

**Feature reuse rate.** Percentage of features used in datasets that were discovered through EFS rather than rebuilt in custom pipelines. Baseline: estimated below 20% across teams. Target: 40% within two quarters of GA. Tracked via feature provenance metadata on every exported dataset.

### ENGAGEMENT METRICS

**Weekly active Model Builders (WAU).** Number of distinct Model Builder users performing at least one feature search or dataset build per week. Target: 50 WAUs within 90 days of GA at the first major customer.

**Dataset build success rate.** Percentage of dataset build attempts that complete without an error on the first try. Target: above 90%. A low rate indicates schema confusion, data quality issues surfaced too late, or UX friction in the builder flow.

**Search-to-select conversion.** Percentage of search sessions that result in at least one feature being added to a dataset. A low rate indicates search relevance issues or a mismatch between the catalog's metadata and what users are looking for.

### QUALITY METRICS

**Model Builder NPS.** In-product NPS survey sent after the first completed dataset export. Target: above 40 at beta. Qualitative feedback from this survey will inform the GA iteration.

**Support ticket deflection.** Reduction in data engineering tickets initiated for the purpose of "what features exist" and "how do I get training data." Target: 50% reduction at accounts where the UI is in use, measured against a pre-launch baseline.

> **GUARDRAIL METRIC**
>
> Dataset correctness rate: the percentage of exported datasets that, when validated by the downstream training pipeline, do not fail due to point-in-time leakage, null feature values caused by incorrect time windows, or schema mismatches. This must remain above 98% at all times. A drop below this threshold triggers an immediate hold on new dataset exports.

## 06 User Personas

This PRD is scoped to the Model Builder as the primary persona. Two supporting personas interact with the same system but are served by surfaces outside this PRD's scope.

### Model Builder — Primary Persona

Data Scientist / ML Engineer · Consumes features, builds training datasets

> *"Give me a trusted, versioned training dataset so I can build a model fast. I shouldn't need to know how the data was engineered or write SQL to figure out what exists."*

**GOALS**

- Discover features relevant to a modelling problem without code
- Understand feature quality — freshness, lineage, who uses it
- Assemble a point-in-time-correct training dataset quickly
- Reproduce a dataset from weeks ago for model debugging
- Hand off a versioned dataset to an ML training pipeline

**FRUSTRATIONS**

- No visual catalog — must know exact feature names upfront
- Filing tickets for basic questions about data availability
- Rebuilding features that likely already exist elsewhere
- Training datasets that fail validation with no clear cause
- No way to check data freshness before committing to a feature

**DAY IN THE LIFE**

- Works in Python notebooks and an ML platform (AI Studio)
- Receives a model brief from a business stakeholder — e.g., improve fraud recall by 5%
- Needs to assemble a training dataset covering customer, transaction, and merchant signals for the last 18 months
- Currently: 2 to 3 days waiting for data engineering support, then several hours of trial-and-error schema inspection
- With EFS UI: browse Fraud Analytics repository → search for relevant features → inspect lineage and freshness → build dataset → export to AI Studio in one session

### Feature Engineer — Supporting Persona

Data Engineer · Builds and publishes features · Largest creator

> *"How do I turn raw data into governed, reusable features that others can trust?"*

The Feature Engineer is the primary producer of features consumed by Model Builders. They author transformations, manage feature pipelines, and own the quality of published features. Their workflows —

pipeline authoring, deployment, and lifecycle management — are out of scope for this PRD but are a dependency: the quality of what they publish directly affects what the Model Builder discovers.

### Platform Administrator — Supporting Persona

Data Platform Owner · Manages repository structure and access

> *"How do I organise EFS so multiple teams can work in parallel and I can keep it under control?"*

The Admin sets up repositories, grants role-based access, and monitors platform health. The Discovery UI inherits the access model the Admin has configured — a Model Builder sees only the domains and features their role permits. Admin-facing configuration surfaces are out of scope for this PRD.

## 07 Jobs to be Done

The following jobs are drawn from Model Builder interviews, customer engagement sessions (Sanofi, BNP Paribas), and prototype validation. Each job is written from the Model Builder's perspective.

**J1** When I start a new modelling project, **I want to browse and search the feature catalog by domain or keyword** so that I can identify relevant signals without knowing exact table names or asking for help.

**J2** When I find a candidate feature, **I want to see its lineage, data freshness, owner, and recent usage by other teams** so that I can decide whether to trust it before including it in my training data.

**J3** When I am composing a dataset, **I want to configure a time window and preview a sample of the dataset with point-in-time correctness enforced** so that I can validate the data makes sense before committing compute to a full dataset build.

**J4** When my dataset is ready, **I want to export it directly to my ML training environment with a versioned reference** so that my model training run is reproducible and the dataset can be audited or reused later.

**J5** When I return to a model I built three months ago, **I want to reload the exact dataset recipe — same features, same time window, same version** so that I can reproduce the original training run and debug a performance regression.

**J6** When I am not sure which features are most relevant to my use case, **I want to describe my modelling goal in natural language and receive AI-assisted feature recommendations** so that I can discover features I would not have found through keyword search alone.

## 08 Solution Overview

The EFS Discovery UI is a web application that exposes the Enterprise Feature Store catalog as a self-service product for Model Builders. It is built on top of the existing EFS Python SDK and does not require changes to the underlying data layer or feature storage infrastructure.

### FIVE CORE SURFACES

**Repository Home.** The entry point. Displays the repositories a user has access to, summary statistics (feature count, domain count, last updated), and quick links to recently visited domains. The home surface anchors the user's mental model — EFS is organised by repository, then by domain, then by feature.

**Domain Browser.** A hierarchical view of all feature domains within a repository (Customer, Transactions, Merchant, Accounts, and others). Each domain card shows the feature count, a brief description, the owning team, and freshness status. Clicking a domain surfaces its features in a sortable list with inline metadata.

**Feature Search.** Full-text and semantic search across feature names, descriptions, tags, and owner metadata. Filters include domain, data type, freshness window, and usage frequency. Results rank by relevance and show a one-line description and freshness indicator at a glance, with a side panel for full detail without leaving the search context.

**Feature Detail Panel.** A slide-over panel showing complete feature metadata: schema, sample values, transformation lineage, owning pipeline, data freshness (last run, SLA status), usage statistics (how many models use it, which teams), and a historical freshness chart. This panel is the trust-building surface — it answers "should I use this feature?" without requiring any code.

**Dataset Builder.** A guided workflow for composing, previewing, versioning, and exporting a training dataset. Users add features from search or domain browsing, configure a label table and time window for point-in-time joins, preview a sample of the resulting dataset, name and version it, and export it to the connected ML training pipeline. Saved datasets are accessible from a personal library for reuse and reproduction.

### ATLAS AI ASSISTANT

Atlas is an embedded AI assistant accessible from every surface via a persistent floating action button. It accepts natural language queries ("which features capture customer payment velocity in the last 30 days?") and returns ranked feature recommendations with a brief rationale for each suggestion. Atlas is built on semantic search over feature metadata embeddings and does not have access to underlying row-level data. It addresses Job J6 and Sanofi's explicitly stated requirement for NL access to the EFS knowledge base.

### INTEGRATION POINTS

The Discovery UI integrates with the EFS Python SDK for all data operations (search, metadata retrieval, dataset assembly, export). RBAC is inherited from the repository-level permissions configured by the Platform Administrator — no separate access model is introduced. Dataset export targets the AI Studio

training pipeline as the primary handoff point, with the exported dataset reference (version ID, feature list, time window configuration) stored as a versioned artifact in EFS.

## 09 Feature Requirements

Requirements are prioritised using MoSCoW. P0 items are required for alpha; P1 items are required for GA; P2 items are targeted for the first post-GA iteration.

**MUST HAVE — P0**

| ID | REQUIREMENT | DESCRIPTION |
|---|---|---|
| R-01 | Repository & Domain Navigation | Users can browse repositories they have read access to and navigate into domains. Each domain displays feature count, description, owning team, and freshness status. |
| R-02 | Full-Text Feature Search | Users can search for features by name, description, or tag across all accessible domains. Results return within 500ms at p95. Filters for domain, data type, and SLA status must be available. |
| R-03 | Feature Detail — Schema & Stats | Each feature has a detail panel showing: column schema, data type, nullable flag, sample values, description, owning pipeline, owner team, and creation date. |
| R-04 | Feature Detail — Lineage | The detail panel shows upstream data sources and transformation steps for each feature, allowing a Model Builder to trace a feature back to its raw data origin. |
| R-05 | Feature Detail — Freshness | The detail panel shows last pipeline run timestamp, configured SLA window, and a pass/warn/fail freshness status. A 30-day freshness history chart is displayed. |
| R-06 | Dataset Builder — Feature Selection | Users can add features to a dataset builder cart from search results or domain views. Features can be removed. The current cart is persistent within a session. |
| R-07 | Dataset Builder — Time Window Config | Users specify a label table, a primary key, a timestamp column for point-in-time joins, and a date range for the dataset. The UI validates configuration before allowing a build. |
| R-08 | Dataset Builder — Point-in-Time Preview | Before committing to a full build, users can preview a sample of up to 1,000 rows with point-in-time correctness applied. Preview renders within 3 seconds for this row count. |
| R-09 | Dataset Export to AI Studio | Users can trigger a full dataset build and export the result to the connected AI Studio training pipeline. A versioned dataset reference (ID, feature list, time window, build timestamp) is stored in EFS. |
| R-10 | RBAC Enforcement | Repository and feature visibility is governed by the access model configured by the Platform Administrator. Users without read |

| ID | REQUIREMENT | DESCRIPTION |
|---|---|---|
| | | permission for a domain cannot see it or search its features. |

### SHOULD HAVE — P1

| ID | REQUIREMENT | DESCRIPTION |
|---|---|---|
| R-11 | Saved Datasets & Version History | Users can name, save, and retrieve dataset recipes. Each saved dataset shows the feature list, time window, build date, and export status. Multiple versions of the same dataset are retained. |
| R-12 | Feature Usage Analytics | The feature detail panel shows usage statistics: number of models that include the feature, teams using it, and trend over the last 90 days. This data directly informs the reuse-vs-rebuild decision. |
| R-13 | Atlas AI Assistant | A natural-language assistant embedded in the UI accepts queries about the feature catalog and returns ranked feature recommendations with rationale. Atlas operates on feature metadata only and has no access to row-level data. |
| R-14 | Semantic Feature Search | In addition to keyword matching, search results are ranked using semantic similarity over feature descriptions and tags. A search for "customer payment velocity" surfaces relevant features even without an exact keyword match. |
| R-15 | Freshness SLA Indicators | Domain and search views display a freshness traffic-light indicator (green/amber/red) for each feature at a glance, without requiring the user to open the detail panel. |

### COULD HAVE — P2

| ID | REQUIREMENT | DESCRIPTION |
|---|---|---|
| R-16 | Dataset Version Diff | Users can compare two versions of the same saved dataset side-by-side to understand which features changed, what the time window delta is, and how build statistics differ. |
| R-17 | Dataset Recipe Sharing | Users can share a read-only link to a saved dataset recipe with teammates, enabling collaboration and reducing duplicate dataset construction across a team. |
| R-18 | Cross-Domain Feature Comparison | Users can pin two or more features side-by-side to compare schema, freshness, lineage, and usage — useful when two similarly named features exist across domains. |

**WON'T HAVE — THIS RELEASE**

| ID | REQUIREMENT | RATIONALE |
|---|---|---|
| R-19 | Feature Authoring | Creating and publishing new features is the Feature Engineer's domain. Introducing authoring in the Model Builder UI creates scope and persona confusion. |
| R-20 | Repository & Access Administration | Admin workflows require a separate surface with different permission requirements. Including them here would increase scope and delay the Model Builder release. |
| R-21 | Online Serving Configuration | Configuring features for real-time inference is a post-training step. It will be addressed in a subsequent release targeting the serving infrastructure. |

## 10 User Stories & Acceptance Criteria

**US-01** — **P0 · MUST**

As a Model Builder, I want to search for features by keyword so that I can find relevant signals without knowing exact feature names or table schemas.

**ACCEPTANCE CRITERIA**

- ✓ Given I type a search term in the search bar, when I press enter or after 300ms of inactivity, then results appear within 500ms showing feature name, domain, and a one-line description.
- ✓ Given results are returned, when I apply a domain filter, then only features from that domain are shown and the count updates immediately.
- ✓ Given I search for a term that matches no features, then the UI shows a clear empty state with a suggestion to try a broader term or use Atlas.
- ✓ Given I click a search result, then the feature detail panel opens without navigating away from the results list.

**US-02** — **P0 · MUST**

As a Model Builder, I want to see a feature's data lineage so that I can verify its data provenance and trust it before including it in a training dataset.

**ACCEPTANCE CRITERIA**

- ✓ Given I open a feature detail panel, then I can see the upstream data source (table name, schema, connection) and the transformation steps applied.
- ✓ Given a feature has multiple upstream sources, then all sources are listed and the transformation logic is shown in a readable format.
- ✓ Given the feature's source pipeline is broken or stale, then the lineage view shows a warning indicator with the last known good run timestamp.
- ✓ Given I click on an upstream source, then I am shown its connection metadata but not redirected to an external system without my action.

**US-03** P0 · MUST

As a Model Builder, I want to preview a point-in-time sample of my composed dataset so that I can validate the data is correct before triggering a full build.

ACCEPTANCE CRITERIA

- ✓ Given I have added at least one feature and configured a label table and time window, then a "Preview" button is enabled in the dataset builder.
- ✓ Given I click Preview, then a sample of up to 1,000 rows is rendered in a table within 3 seconds, with point-in-time correctness applied (no future feature values leak into past label rows).
- ✓ Given the preview contains null values for a feature column, then the null percentage is shown in the column header.
- ✓ Given the time window configuration is invalid (e.g., end date before start date), then the Preview button is disabled and a validation message explains the error.

**US-04** P0 · MUST

As a Model Builder, I want to export a completed dataset to my AI Studio training pipeline so that I can start model training immediately with a versioned, reproducible reference.

ACCEPTANCE CRITERIA

- ✓ Given my dataset is previewed and valid, when I click "Build & Export", then the system triggers a full dataset build and shows a progress indicator with estimated completion time.
- ✓ Given the build completes successfully, then a versioned dataset artifact is created in EFS containing the feature list, time window configuration, row count, and build timestamp.
- ✓ Given the build completes, then the dataset reference is automatically surfaced in the connected AI Studio training environment.
- ✓ Given the build fails, then I receive a clear error message with the failure reason and a link to the relevant pipeline logs, without losing my dataset configuration.

**US-05** P1 · SHOULD

As a Model Builder, I want to save and reload a dataset recipe so that I can reproduce a past training run or reuse a validated feature set for a new model iteration.

ACCEPTANCE CRITERIA

- ✓ Given I have built a dataset, when I name and save it, then it appears in my Saved Datasets list with feature count, time window, and build date.
- ✓ Given I open a saved dataset, then I can reload the exact configuration — same features, same time window — into the builder without reselecting anything manually.
- ✓ Given I modify a saved dataset and save again, then a new version is created and the previous version remains accessible.
- ✓ Given I have no saved datasets, then the Saved Datasets view shows an empty state with a call to action to build a first dataset.

**US-06** P1 · SHOULD

As a Model Builder, I want to ask Atlas in natural language which features are relevant to my modelling goal so that I can discover features I would not have found through keyword search alone.

ACCEPTANCE CRITERIA

- ✓ Given I open the Atlas panel and type a natural language query, then Atlas returns a ranked list of feature recommendations within 2 seconds, each with a brief rationale.
- ✓ Given Atlas returns recommendations, then I can add any recommended feature directly to the dataset builder from within the Atlas panel without leaving my current view.
- ✓ Given Atlas returns no confident recommendations, then it says so clearly and suggests alternative search terms rather than showing low-confidence results.
- ✓ Given I ask Atlas a question outside its knowledge scope (e.g., about a feature that does not exist), then Atlas responds gracefully and does not fabricate feature metadata.

**US-07** P0 · MUST

As a Model Builder, I want to see a feature's freshness status at a glance so that I do not inadvertently include stale data in a training dataset.

ACCEPTANCE CRITERIA

- ✓ Given I am browsing a domain or viewing search results, then each feature row shows a freshness indicator (green for within SLA, amber for approaching SLA, red for breached SLA).
- ✓ Given I open a feature detail panel, then I see the last pipeline run timestamp, configured SLA window, and a 30-day freshness history chart.
- ✓ Given a feature is in a red (SLA-breached) state, then it is visually distinguished in the dataset builder and a warning is shown before I can include it in an export.
- ✓ Given freshness data is unavailable for a feature (e.g., the pipeline has never run), then the indicator shows "Unknown" rather than defaulting to a green state.

**US-08** P0 · MUST

As a Model Builder, I want to see which other models and teams are using a feature so that I can gauge its reliability and avoid duplicating proven feature work.

ACCEPTANCE CRITERIA

- ✓ Given I open a feature detail panel, then I see a usage count showing how many distinct datasets have included this feature in the last 90 days.
- ✓ Given usage exists, then I can see the names of the teams (not individual users) that have used the feature, respecting any team-level visibility restrictions.
- ✓ Given a feature has never been used in any dataset, then the usage section explicitly shows "Not yet used in any dataset" rather than zero, to clarify the signal.

## 11 Non-Functional Requirements

| CATEGORY | REQUIREMENT | TARGET |
|---|---|---|
| **Performance** | Feature search results must return for 95% of queries without exceeding the latency target under normal catalog load (up to 10,000 features). | p95 < 500ms |
| **Performance** | Dataset preview (1,000 rows, point-in-time applied) must complete within the target window to maintain an interactive workflow. | < 3 seconds |
| **Scalability** | The catalog and search surfaces must remain performant as the feature count grows, with no UI degradation from pagination or client-side rendering limits. | 10,000+ features |
| **Scalability** | The system must support concurrent active Model Builder sessions without degradation. | 50 concurrent users |
| **Availability** | The Discovery UI must meet enterprise SLA standards for uptime, excluding planned maintenance windows. | 99.9% uptime |
| **Security** | RBAC is inherited from the EFS repository-level access model. No user sees a feature, domain, or repository they do not have explicit read permission for. There is no way to infer the existence of restricted features through search result counts or error messages. | Zero information leakage across tenants |
| **Security** | Dataset preview exposes a row-limited sample only. Full row-level data is never surfaced in the browser; all data access goes through the EFS SDK's access-controlled pipeline. | Max 1,000 preview rows |
| **Accessibility** | The UI must conform to Web Content Accessibility Guidelines at the AA level. All interactive elements must be keyboard-navigable and screen-reader compatible. | WCAG 2.1 AA |
| **Auditability** | Every dataset build and export action is logged with user identity, timestamp, feature list, and time window configuration. Logs are immutable and retained for compliance purposes. | Full audit trail, 2-year retention |
| **Internationalisation** | The UI is built to support localisation (i18n). The initial release ships in English; the architecture must not hard-code strings in a way that prevents future locale support. | i18n-ready |

## 12 Out of Scope

The following areas are explicitly excluded from this PRD. They may be addressed in future releases or separate product tracks.

**Feature authoring and pipeline management.** Creating, editing, deploying, or monitoring feature transformation pipelines is the Feature Engineer's domain. This PRD covers feature consumption only.

**Platform and repository administration.** Repository creation, team and user access management, domain schema governance, and SLA threshold configuration are addressed in a separate Admin PRD.

**Online feature serving.** Configuring features for real-time inference, managing online store deployments, and monitoring inference latency are post-training concerns. They are tracked on a separate roadmap item.

**External ML platform integrations.** Native connectors to SageMaker, Vertex AI, Azure ML, or Databricks are not in scope. The initial export target is AI Studio only.

**Python SDK changes.** The Discovery UI is built on the existing SDK. New SDK endpoints or breaking SDK changes required to support the UI are tracked as engineering dependencies, not product requirements in this document.

**Mobile and native application surfaces.** The Discovery UI is a web application optimised for desktop screen widths. Mobile-optimised views are not in scope for the initial release.

**Data quality remediation.** The UI surfaces data quality indicators (freshness, null rates in preview) but does not provide tooling to fix quality issues. Remediation remains with the Feature Engineer and their pipeline tools.

## 13 Dependencies & Risks

### KEY DEPENDENCIES

**EFS Python SDK stability.** The Discovery UI calls EFS SDK endpoints for search, metadata retrieval, dataset assembly, and export. Any breaking changes to the SDK API during the build phase will require UI rework. Engineering alignment on an API stability commitment is a prerequisite for starting UI development.

**Semantic search infrastructure.** Requirements R-14 and R-13 (Atlas) depend on a vector index built over feature metadata. This index does not exist today. Building it is a data engineering dependency that must be scoped and resourced separately.

**AI Studio export API.** Dataset export (R-09) requires an API contract with the AI Studio training pipeline. This contract must be defined and agreed before the builder surface can be fully implemented.

**Feature metadata completeness.** The value of discovery is directly proportional to the quality of feature descriptions, tags, and lineage metadata in the catalog. If Feature Engineers have not populated metadata, search and Atlas will return poor results. A metadata enrichment campaign is a pre-launch prerequisite.

### RISK REGISTER

| RISK | LIKELIHOOD | IMPACT | MITIGATION |
|---|---|---|---|
| **SDK breaking changes** during the build phase invalidate UI data contracts mid-development. | MEDIUM | HIGH | Agree a versioned API contract and a freeze period for SDK endpoints consumed by the UI. Include contract test coverage in CI. |
| **Low feature metadata quality** makes search irrelevant and reduces Atlas recommendation quality at launch. | HIGH | HIGH | Run a structured metadata enrichment sprint with the Feature Engineer team at the alpha customer (Sanofi) 4 weeks before alpha launch. Define minimum metadata bar (description required, tags required) as a publish-time gate. |
| **Point-in-time correctness bugs** cause training data leakage that is not caught until post-model deployment. | MEDIUM | HIGH | Treat PIT correctness as a guardrail metric. Build automated leakage detection tests into the dataset build pipeline. Require explicit sign-off from the data engineering team on the PIT implementation before alpha. |
| **Atlas recommendation quality** too low to be useful, eroding trust in the feature and in the product overall. | MEDIUM | MEDIUM | Gate Atlas behind a P1 priority (not required for alpha). Set an internal quality bar (top-3 precision above 60% on a held-out evaluation set) before enabling it in GA. Provide graceful fallback to keyword search when Atlas confidence is below threshold. |
| **Model Builder adoption lower than expected** | MEDIUM | MEDIUM | Design the UI to produce the same Python SDK call that the user would have written — show it |

| RISK | LIKELIHOOD | IMPACT | MITIGATION |
| --- | --- | --- | --- |
| because users default to existing Python workflows out of habit. | | | as a "copy to notebook" option after every dataset build. Make the UI a complement to the SDK, not a replacement. Run hands-on onboarding sessions at alpha customers. |
| **Semantic search infrastructure** not resourced or deprioritised, blocking R-14 and Atlas. | LOW | MEDIUM | Decouple vector indexing from the core search path. Ship keyword search (R-02) for alpha without semantic ranking. Semantic layer is a progressive enhancement, not a blocker. |

## 14 Go-to-Market & Launch Plan

**ALPHA · WEEKS 1–6**

### Closed Alpha with Sanofi

Deploy P0 requirements (R-01 through R-10) to a single internal Sanofi environment. The goal is to validate core workflows — discovery, detail, builder, export — with 5 to 8 Model Builder users under PM-led observation. Success criteria: 3 successful end-to-end dataset builds with no data engineering involvement. Feedback collected weekly through structured interviews and in-product event logs.

**BETA · WEEKS 7–14**

### Limited Beta with BFSI Cohort

Expand to BNP Paribas and one additional BFSI customer with P1 requirements (R-11 through R-15) enabled. Target 20 to 30 WAU across both accounts. Run in-product NPS survey at the end of week 10. Success criteria: NPS above 40, T2TD under 30 minutes at median, search-to-select conversion above 50%. Fix P0 issues surfaced in alpha before this phase begins.

**GA · WEEK 15**

### General Availability — Horizon-2 Release

Bundle the Discovery UI with the Horizon-2 platform release. All P0 and P1 requirements ship. P2 requirements are on the post-GA roadmap. GA is gated on the guardrail metric (dataset correctness rate above 98%) being met in beta, and on the security review passing. Communicate GA through customer release notes, a PM-led demo webinar, and updated EFS product documentation.

**POST-GA · WEEKS 16+**

### P2 Iteration & Expansion

Prioritise P2 requirements (R-16 through R-18) based on feedback from GA cohort. Begin scoping the Feature Engineer surface as a separate PRD. Review semantic search infrastructure investment based on Atlas adoption data from GA. Target the Healthcare vertical (fastest-growing segment) for next expansion.

#### ENABLEMENT PLAN

**In-product onboarding.** A first-use guided tour covering the five core surfaces. Dismissible but re-accessible from the help menu. Available from alpha.

**Documentation.** User guide covering feature search, the dataset builder, and Atlas. Published to the EFS documentation site at GA. A quick-start card (one A4 page) available for workshop sessions.

**Demo webinar.** A 45-minute PM-led live demo with Q&A, run the week of GA. Recorded and distributed to the customer success team for ongoing use.

**Customer success hand-off.** A structured playbook for the customer success team covering the Model Builder onboarding flow, how to identify healthy adoption signals in product analytics, and escalation paths for quality issues.

## 15 Open Questions

| # | QUESTION | OWNER | STATUS |
|---|---|---|---|
| OQ-01 | Should dataset export to AI Studio be synchronous (user waits in-UI) or asynchronous (notification when ready)? For large datasets, synchronous is not viable, but for alpha-scale data it may be acceptable. The answer determines the build progress UX. | Engineering | OPEN |
| OQ-02 | What is the right unit of access for saved datasets — personal (visible only to the creator) or team-scoped (visible to all members of the same repository)? This affects the collaboration model and the RBAC implementation for R-11. | PM + Legal | OPEN |
| OQ-03 | Should Atlas be powered by an Anthropic-hosted model, a self-hosted LLM for data-residency-sensitive customers (e.g., Sanofi), or a rule-based semantic search as a fallback? This has significant infrastructure and cost implications. | Arch + PM | OPEN |
| OQ-04 | What is the maximum preview row count that can be served within the 3-second SLA at the expected catalog scale (10,000+ features)? The 1,000-row target in R-08 is based on an estimate and needs validation against the EFS query engine. | Engineering | OPEN |
| OQ-05 | How should the UI handle a feature that is in the catalog but whose source pipeline is permanently decommissioned? Should it remain discoverable with a deprecation warning, or be hidden from search results? | PM + Data Eng | OPEN |
| OQ-06 | For the BFSI segment (Sanofi, BNP Paribas), are there data residency requirements that prevent feature metadata (descriptions, tags, usage statistics) from being sent to an external embedding service for semantic indexing? This may require an on-premises vector index deployment. | Legal + Eng | NEEDS INPUT |
| OQ-07 | What metadata fields are required at feature publish time to meet the minimum bar for useful search and Atlas recommendations? The current EFS SDK has no mandatory description field. A decision here affects the Feature Engineer workflow (out of scope for this PRD but a dependency). | PM + Feature Eng | PROPOSAL READY |
| OQ-08 | Should the "copy to Python" feature (which renders the dataset build as EFS SDK Python code) be included in the alpha release or deferred to GA? It reduces the barrier to hybrid usage but adds scope to the builder surface. | PM | OPEN |